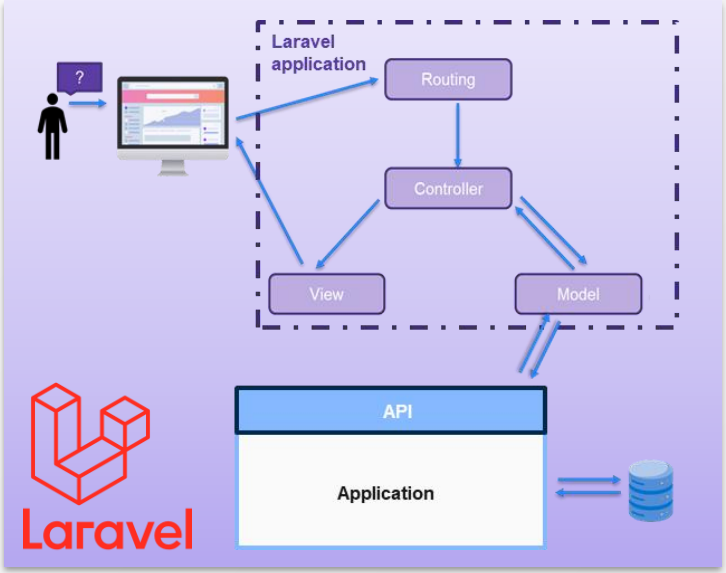
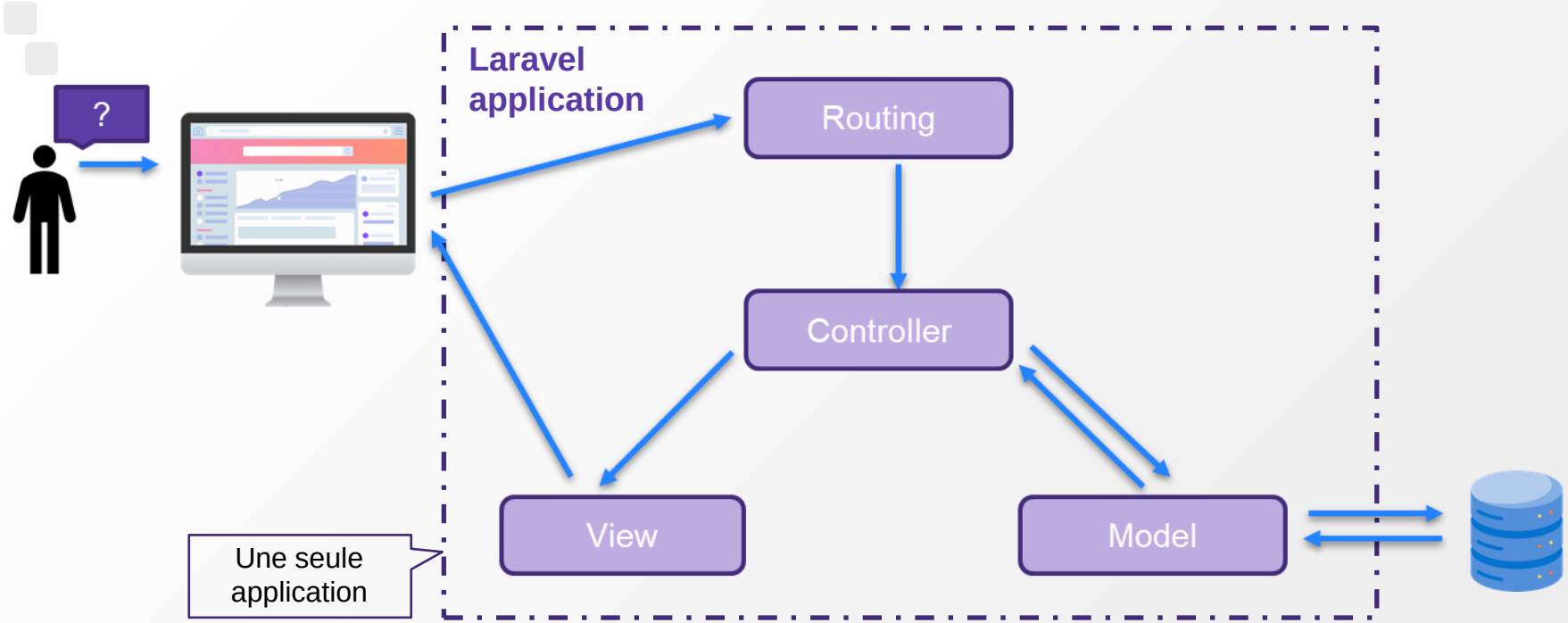
Partie 5



11/03/2026

Développer un site avec Laravel

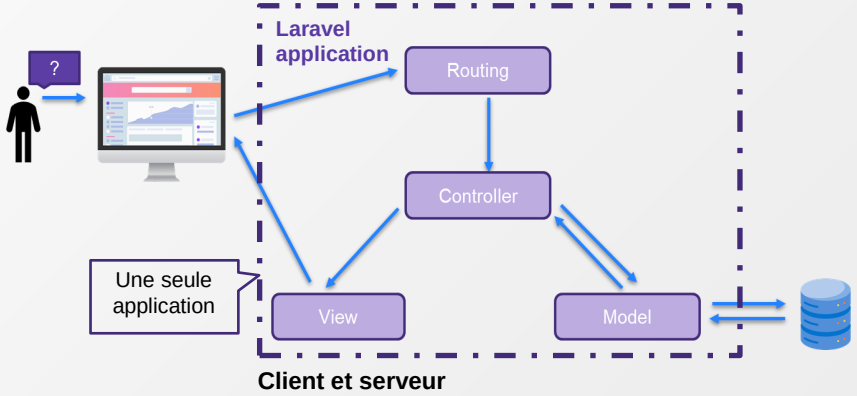


Client et serveur

Développer un site avec Laravel

Dans l'app Laravel développée, le client et le serveur faisaient partie de la même application :

1. Serveur communique directement avec la BDD pour manipuler des données
2. Serveur transmet ces données aux vues via le MVC
3. Le client affiche ces données.



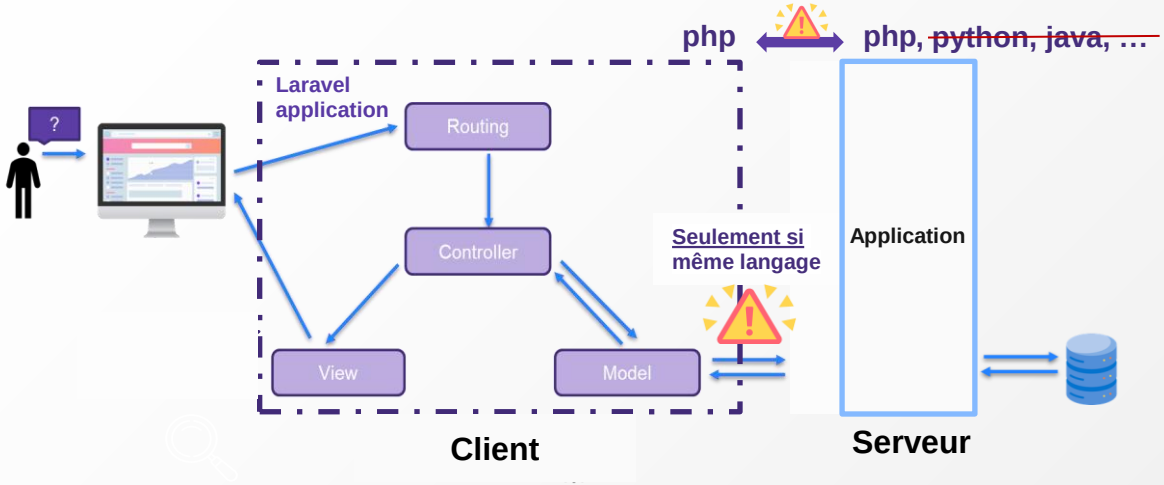
- Réutilisation difficile des données dans d'autres apps
- Couplage fort client / serveur
- Application complexe

Maintenance
Evolution
Changement de technos

- Une seule structure
- Pas de séparation entre client/serveur
- Tout avec le même langage → **php**

Développer un site avec Laravel

Séparation Client / Serveur

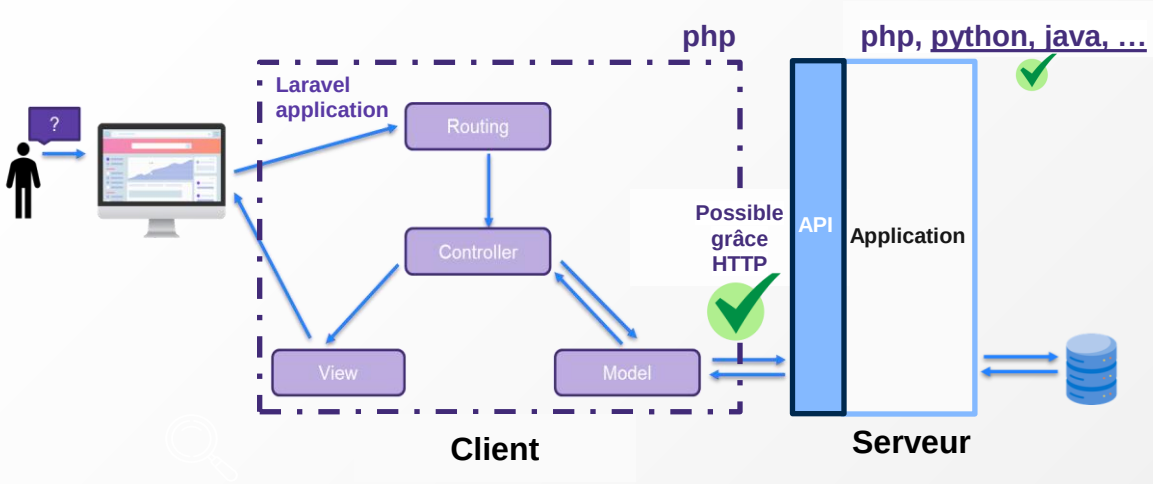


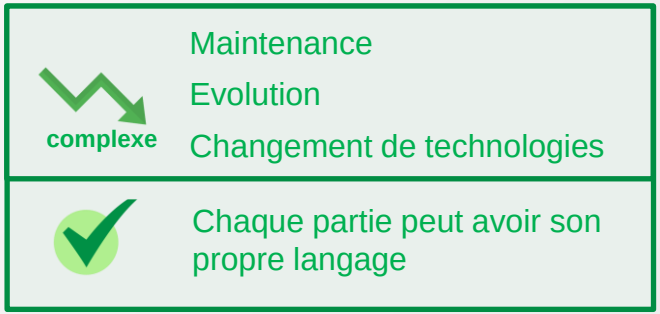
Séparation du serveur dans une application différente :

- Client : application frontend (web, mobile, desktop) **dans notre cas : client → Application laravel php**
- Serveur : application qui communique avec BDD **serveur → Application php**

Développer un site avec Laravel

Séparation Client / Serveur avec API





Séparation du serveur dans une application différente en utilisant :

- Client : n'importe quelle application frontend (web, mobile, desktop) qui **consomme l'API** → **App laravel php**
- Serveur : application qui communique avec BDD et **expose API** → **Application php, python, java, …**

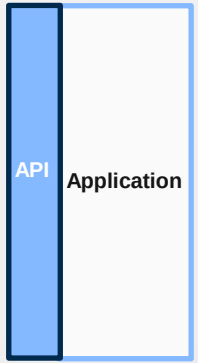
Développer une API

Pourquoi utiliser des APIs

API « Application Programming Interface »

API est une interface permettant à des applications de communiquer entre elles

Non graphique

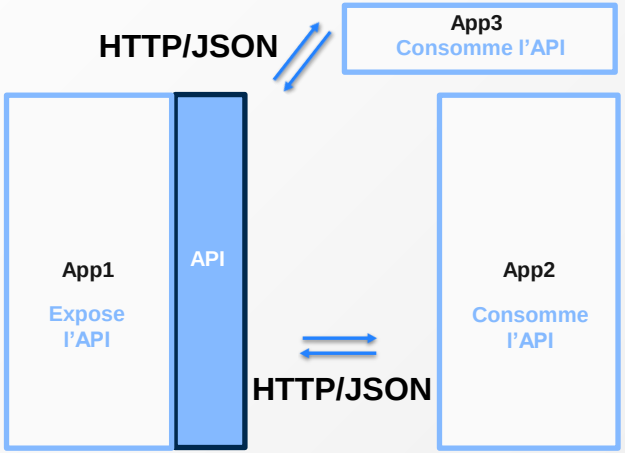


Objectif

- Fournir un accès aux données d'une application
- Transférer des données entre plusieurs applications
- Filtrer et structurer les données avant de les envoyer
- Masquer la logique interne de l'application
- Permettre la réutilisation des services (mobile, web, autre application)

Développer un site avec Laravel

Explication API



- API est un **middleware** entre deux applications homogènes / hétérogènes

  mêmes / différents langages et technologies

- Doit être :

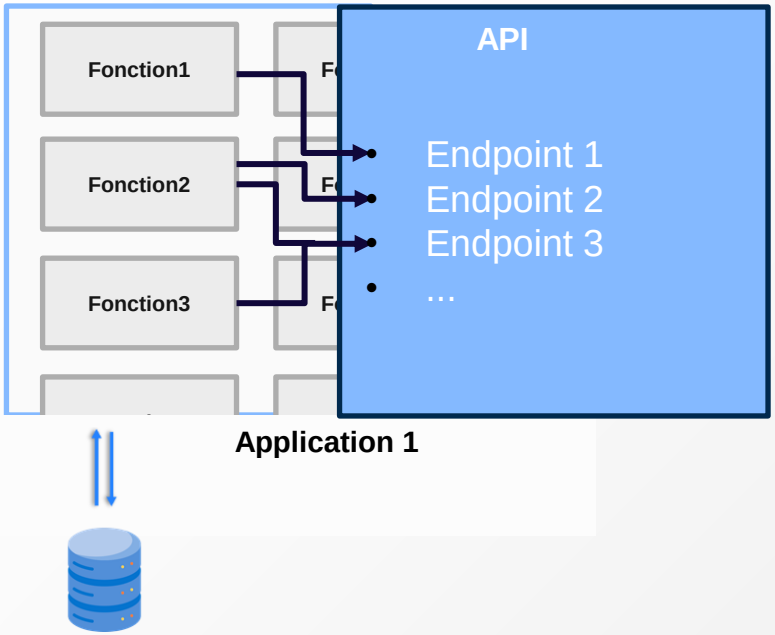
  - **Exposée** par une application → **cette application pourra exposer des données via l'API**
  - **Consommée** par une ou plusieurs applications → **cette application pourra récupérer les donnée à travers l'Api**

- Permet la **communication** entre ces applications en utilisant des protocoles compréhensibles par tous

  HTTP, JSON

Développer une API

Application expose une API

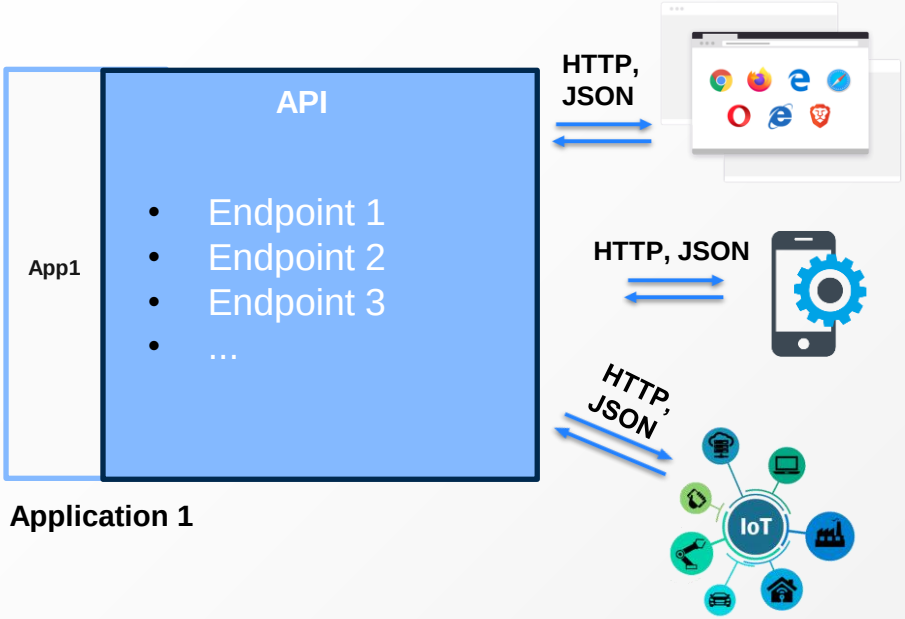


L'entité manipulé par l'API (Articles, magasins, …)

- Une application expose une API
- L'API expose des ressources via plusieurs endpoints
- L'application partage ses données grâce aux endpoints
- **Endpoint** : **URI + méthode HTTP** que le client appelle pour **exécuter une action** sur une ressource
- Chaque endpoint :
  - est liée à une fonction dans le code de l'application
  - permet d'agir sur une ressource

Développer une API

Application consomme une API



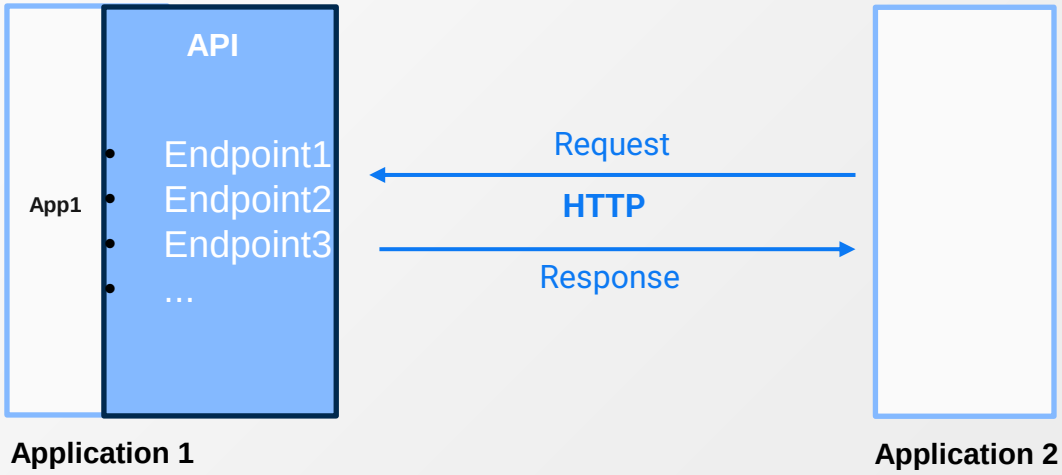
- Toute application qui peut envoyer des requêtes HTTP peut consommer une API

- L'application qui consomme l'API :
  - Envoie des requettes HTTP pour CRUD des données
  - Ne connait pas la logique interne de l'application 1
  - Le client appelle un **endpoint** pour effectuer une action sur les données d'une ressource

Développer une API

Fonctionnement d'une API

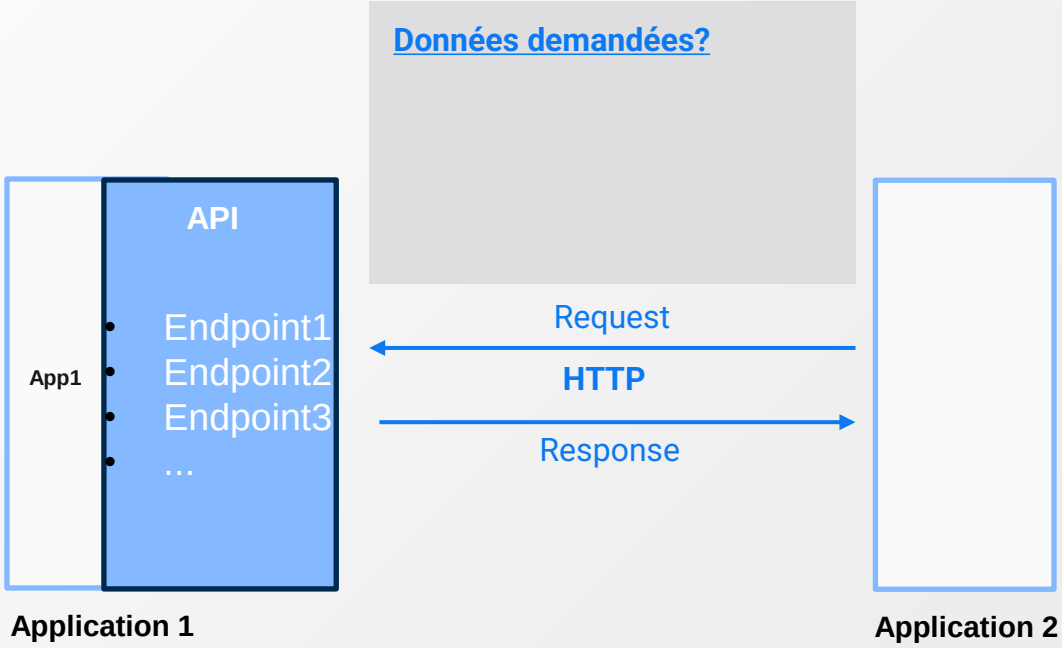
- **Demande (Request)**
- **Endpoint**
- **Réponse (Response)**
- Ressources
- URI
- Méthodes
- Paramètres
- Statut de la réponse (Status)
- Format JSON



Développer une API

Fonctionnement d'une API

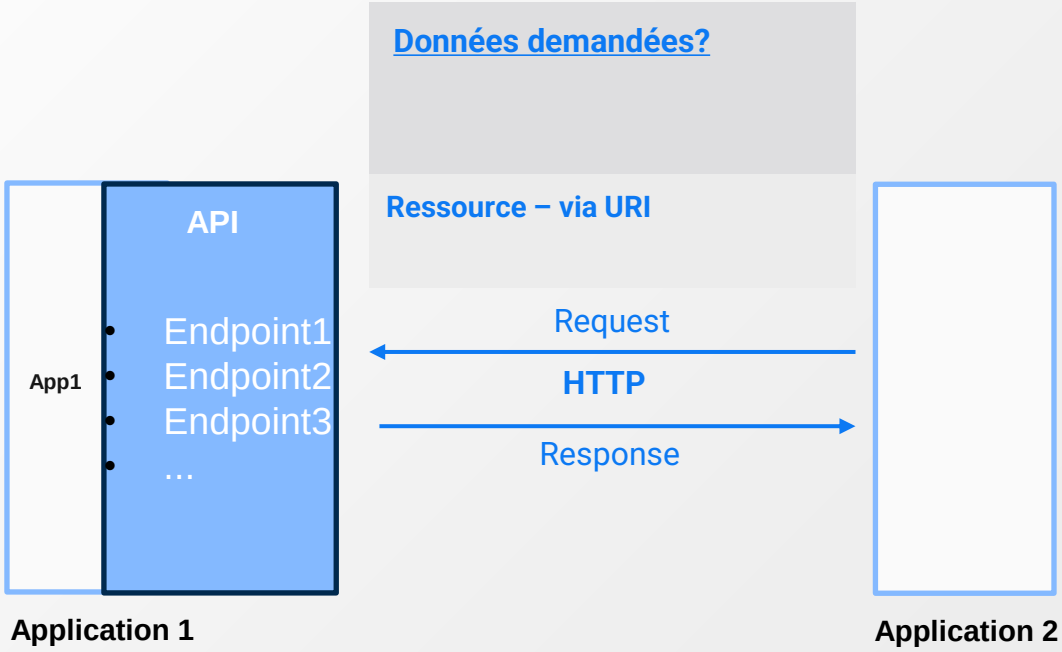
- **Demande (Request)**
- **Endpoint**
- **Réponse (Response)**
- Ressources
- URI
- Méthodes
- Paramètres
- Statut de la réponse (Status)
- Format JSON



Développer une API

Fonctionnement d'une API

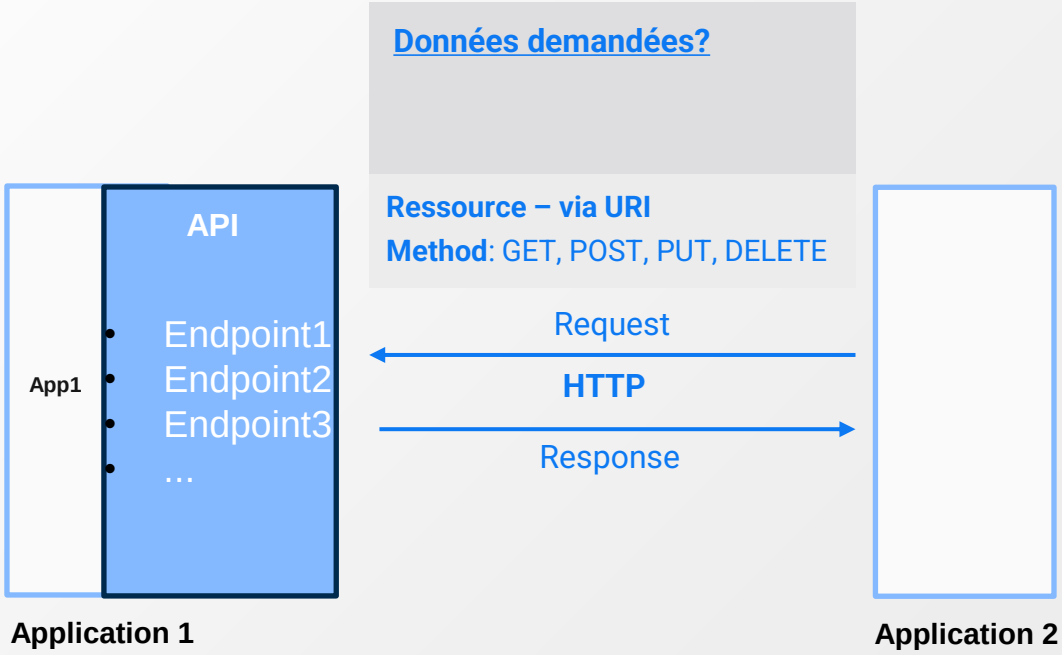
- **Demande (Request)**
- **Endpoint**
- **Réponse (Response)**
- **Ressources**
- **URI**
- Méthodes
- Paramètres
- Statut de la réponse (Status)
- Format JSON



Développer une API

Fonctionnement d'une API

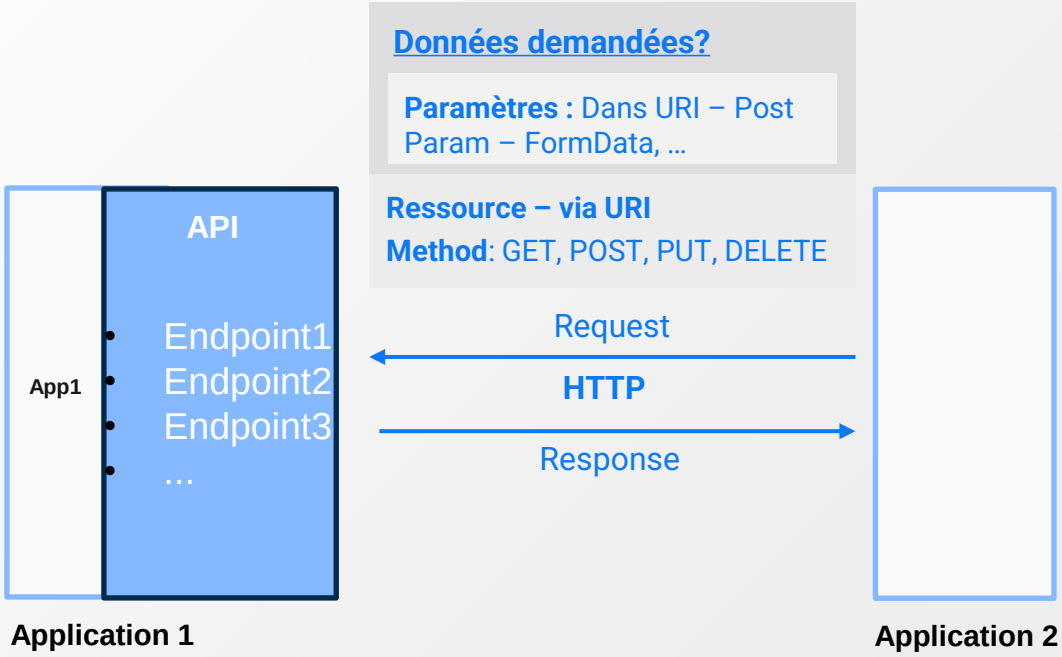
- **Demande (Request)**
- **Endpoint**
- **Réponse (Response)**
- **Ressources**
- **URI**
- **Méthodes**
- Paramètres
- Statut de la réponse (Status)
- Format JSON



Développer une API

Fonctionnement d'une API

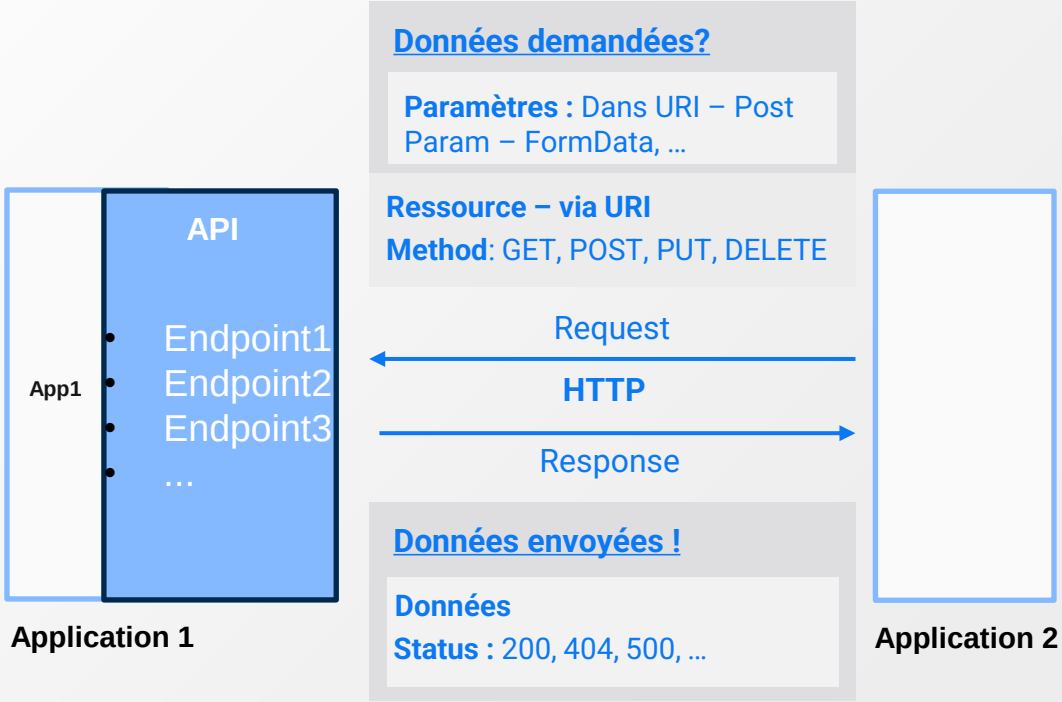
- **Demande (Request)**
- **Endpoint**
- **Réponse (Response)**
- **Ressources**
- **URI**
- **Méthodes**
- **Paramètres**
- Statut de la réponse (Status)
- Format JSON



Développer une API

Fonctionnement d'une API

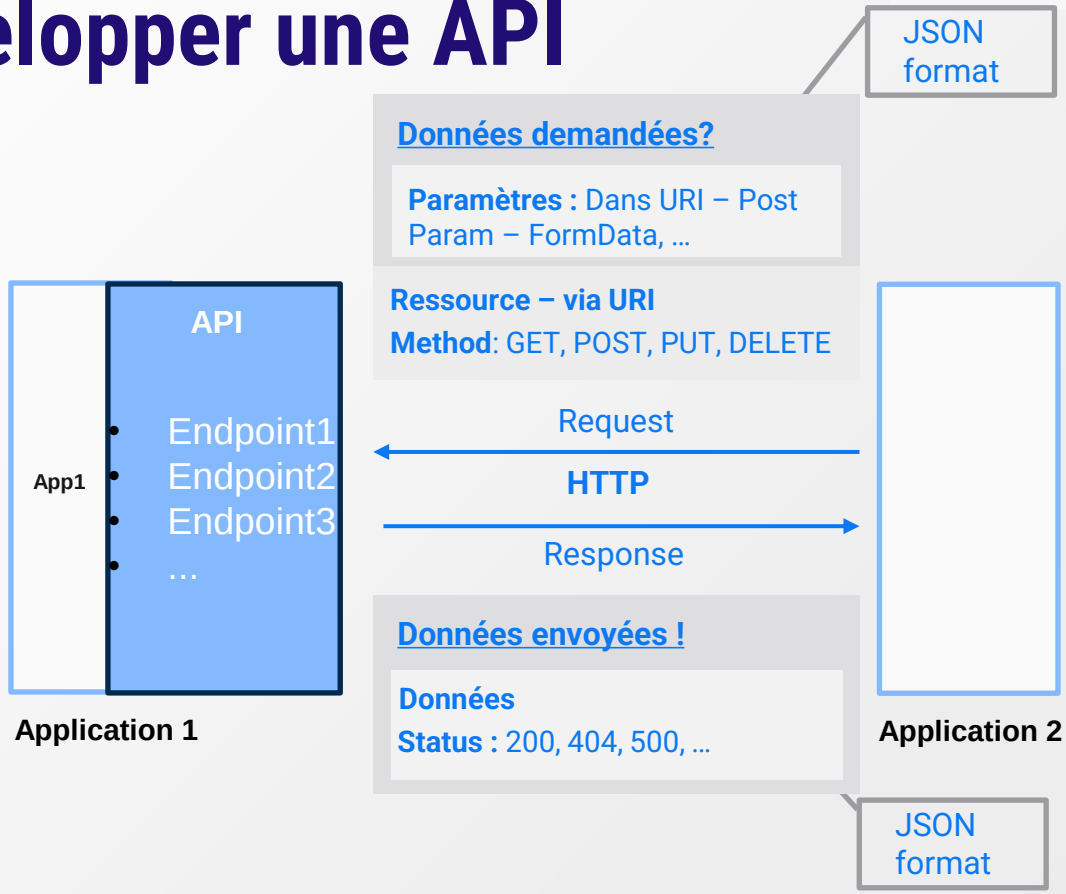
- **Demande (Request)**
- **Endpoint**
- **Réponse (Response)**
- **Ressources**
- **URI**
- **Méthodes**
- **Paramètres**
- **Statut de la réponse (Status)**
- Format JSON



Développer une API

Fonctionnement d'une API

- Demande (Request)
- Endpoint
- Réponse (Response)
- Ressources
- URI
- Méthodes
- Paramètres
- Statut de la réponse (Status)
- Format JSON

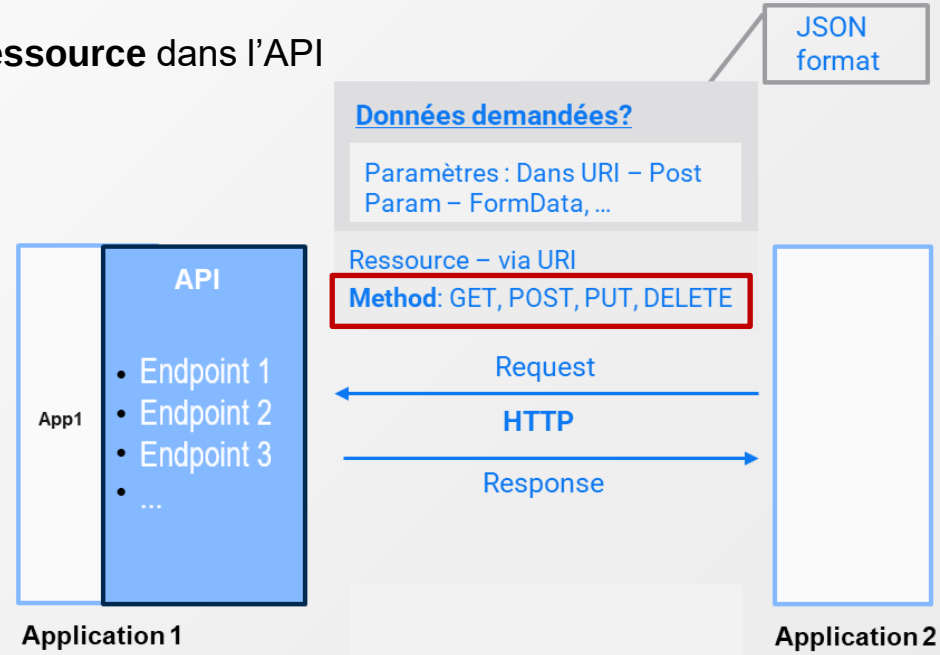


Développer une API

Méthodes HTTP

Chaque méthode correspond à une **action sur une ressource** dans l'API

GET	Lire / récupérer des données
POST	Créer une nouvelle ressource
PUT	Mettre à jour complètement une ressource
PATCH	Mettre à jour partiellement une ressource
DELETE	Supprimer une ressource

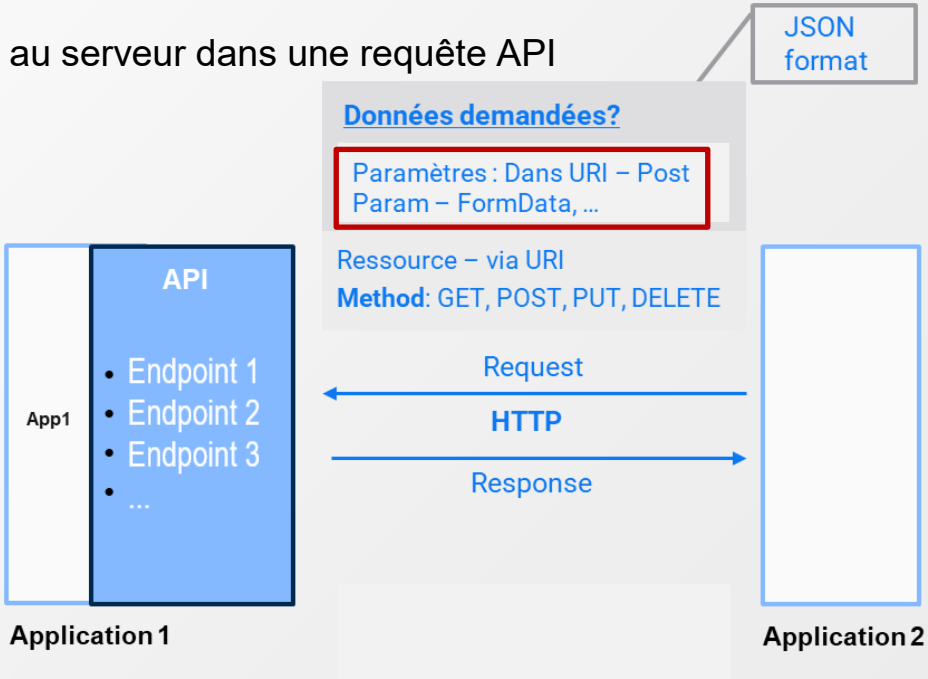


Développer une API

Paramètres d'une API

Les paramètres permettent d'envoyer des informations au serveur dans une requête API

Path Parameter	Identifie une ressource dans l'URL
Query Parameter	Filtre ou recherche des données
Body Parameter	Envoie des données au serveur (Format JSON)
Header Parameter	Informations de configuration ou d'authentification
Form-Data	Envoie des données de formulaire ou fichiers

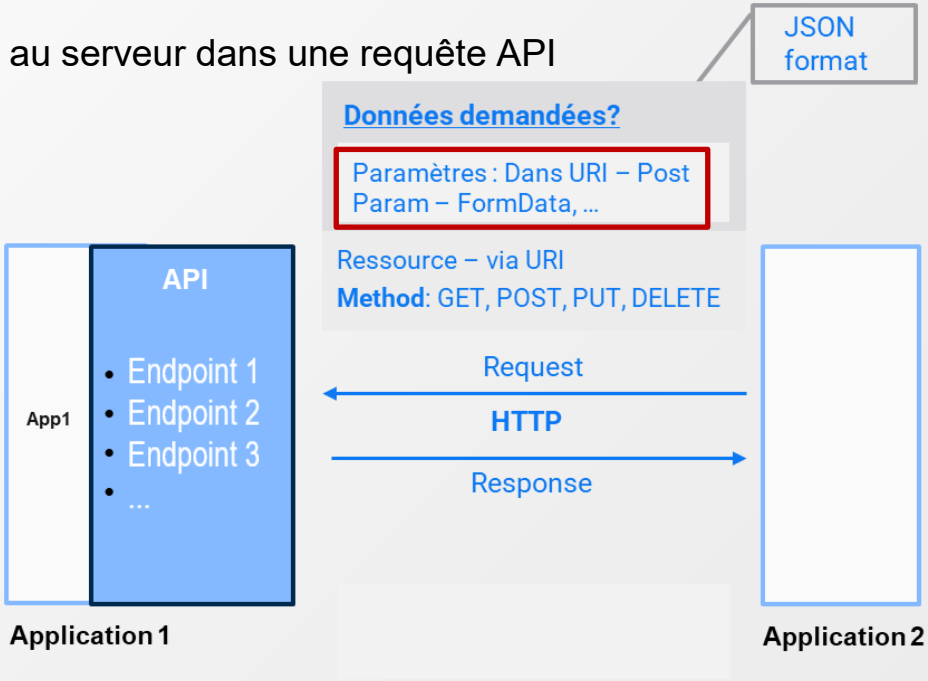


Développer une API

Paramètres d'une API

Les paramètres permettent d'envoyer des informations au serveur dans une requête API

Path Parameter	/users/**10**
Query Parameter	/users**?age=25**
Body Parameter	données JSON envoyées
Header Parameter	Authorization, Content-Type
Form-Data	file: image.png name: document



Développer une API

HTTP Status Codes

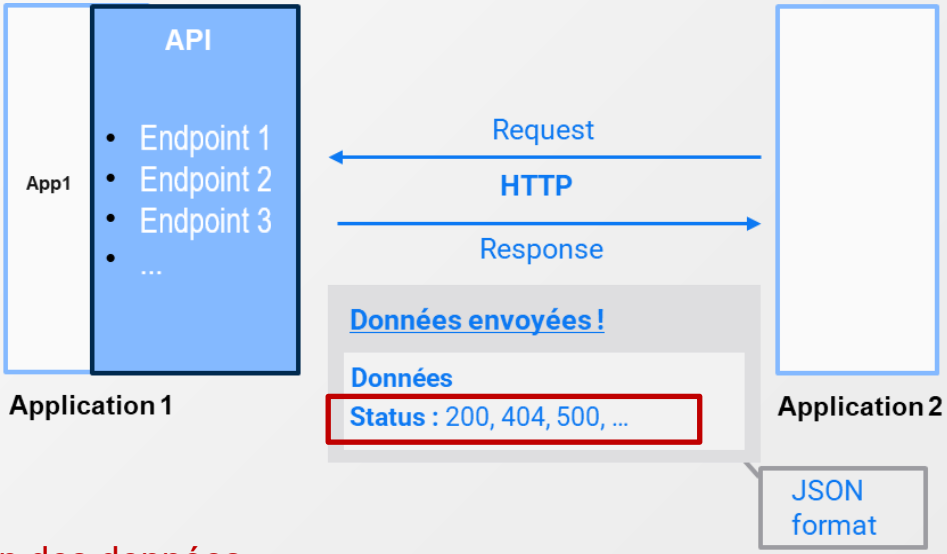
Les HTTP Status Codes indiquent le résultat d'une requête API

2xx – Succès : La requête a réussi

- 200 OK → requête réussie
- 201 Created → ressource créée
- 204 No Content → succès sans contenu

4xx – Erreur client : La requête contient une erreur

- **400 Bad Request** → requête invalide
- **401 Unauthorized** → authentification requise
- **403 Forbidden** → accès refusé
- **404 Not Found** → ressource introuvable
- **422 Unprocessable Entity** → Erreur de validation des données



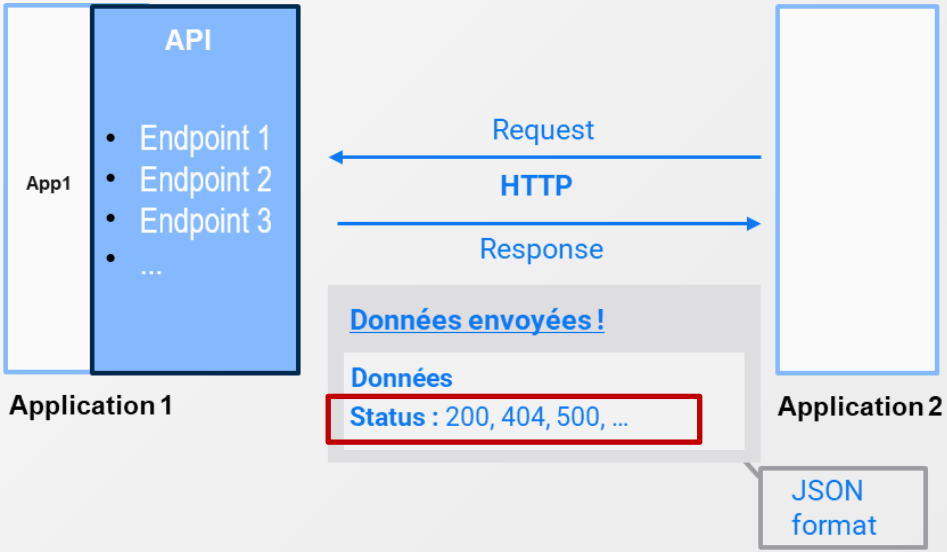
Développer une API

HTTP Status Codes

Les HTTP Status Codes indiquent le résultat d'une requête API

5xx – Erreur serveur : Le serveur a rencontré un problème

- **500 Internal Server Error → erreur serveur**
- **503 Service Unavailable → serveur indisponible**



Développer une API

Exemple endpoints API

Voici un exemple de endpoints exposés par une API que l'on peut concevoir :

Fonctionnalité	Endpoint	Méthode	Paramètres
Lister tous les articles	/api/articles	GET	Aucun
Afficher un article spécifique	/api/articles/{id}	GET	id (ex: 1)
Créer un article	/api/articles	POST	{ "nom": "laptop", "description": "description laptop"}
Mettre à jour un article	/api/articles/{id}	PUT	{ "description": "macbook"}

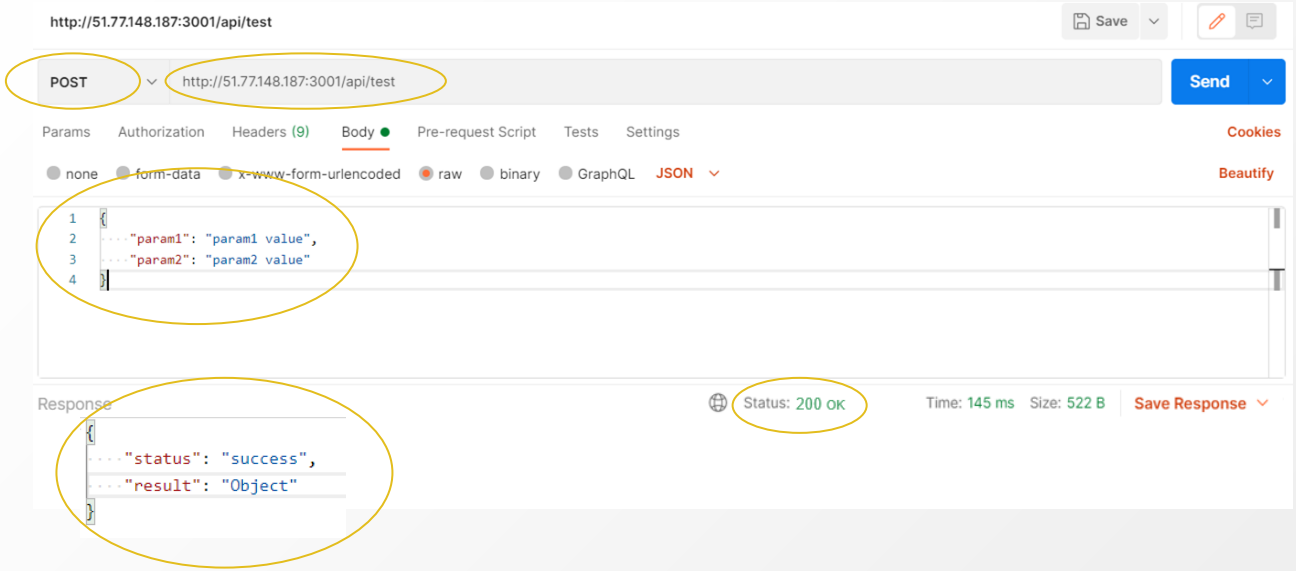
Les endpoints doivent généralement être structurés par ressources (Articles, Magasins…)

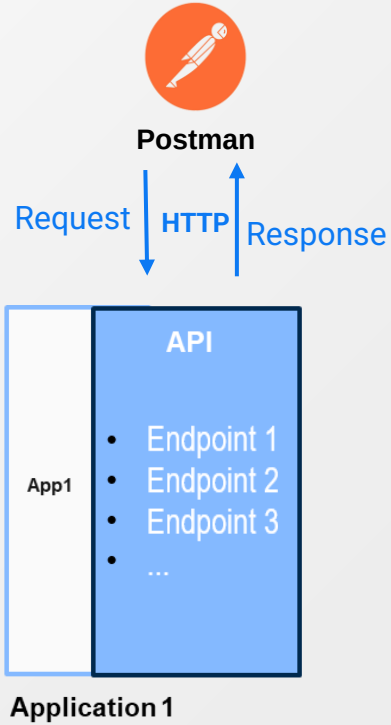
Le client se réfère aux endpoints pour savoir comment manipuler les données souhaitées

Développer une API

Comment tester API ?

Postman est utilisé pour tester des API





Développer une API

Comment documenter API ?

Postman est utilisé pour tester des API

URL

GET | http://51.77.148.187:3000/api/getHumidityData/{id}

Auth. Param.

Params | Authorization | Headers (10) | Body | Pre-request Script | Tests | Settings | Cookies

Type: Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about authorization

Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborative environment, we recommend using variables. Learn more about variables

Username: Username

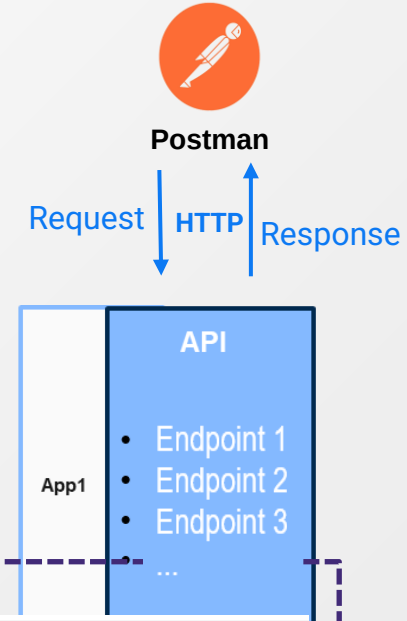
Password: Password

Show Password

JSON

```
{
    "method": "NeuralNetwork",
    "url": "https://xxxxxx.test.csv",
    "testsize": 0.1,
    "independent_variables": ["datetime", "echauffement", "PS_atteinte", "heure", "hm", "month"],
    "dependent_variable": ["temperature"]
}
```

Param en Postman



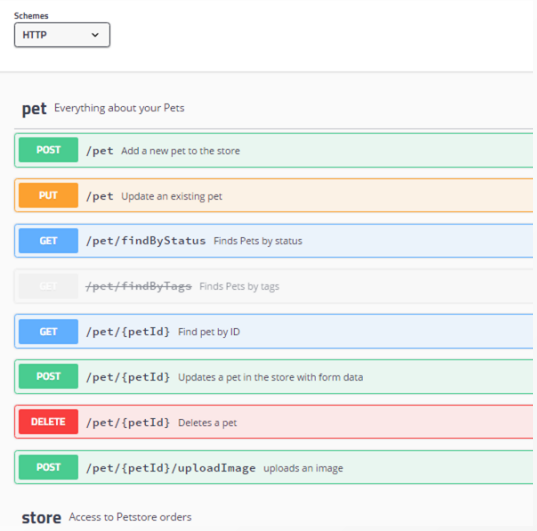
Form-data

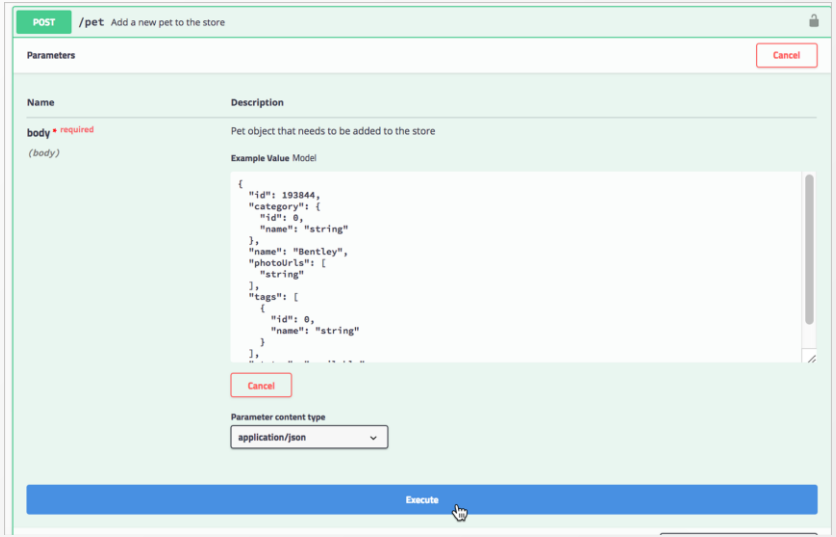
method	NeuralNetwork
training_dataset	hiver1.json
testsize	0.1
independent_variables	["datetime", "echauffement", "PS_atteinte", "heure", "hm"]
dependent_variable	["temperature"]

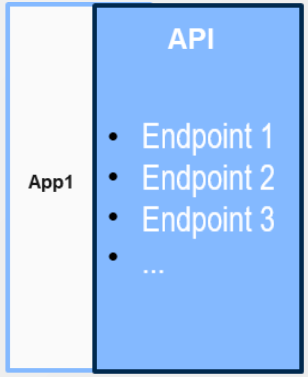
Développer une API

Comment tester API ?

Swagger peut être utilisé pour documenter une API







Application 1

Développer une API

REST API et autres

Type d'API	Principe
REST API	Utilise HTTP (GET, POST, PUT, DELETE) pour accéder aux ressources
SOAP API	Utilise XML et un protocole strict pour échanger des messages
GraphQL API	Le client demande exactement les données nécessaires
gRPC	Communication rapide entre services avec protocole binaire

Dans cette présentation, c'est l'API REST que nous avons vue